Divide and Conquer

Divide and Conquer

- Principle
  - Many algorithms have a recursive structure
    - To solve a given problem, the algorithm uses itself to solve sub-problems with small size, finally combining the results to obtain the solution
  - Three steps
    - **Divide**: divide the problem into sub-problems
    - **Conquer**: solve sub-problems recursively, if sub-problems are small enough then solve directly
    - **Combination**: the results of the sub-problems are the solutions to the given problem

Divide and conquer

- General structure

```
divide-and-conquer(x: problem): solution
begin
   if (x is small and simple) then
            return(solution)
   else
   divide x into many sub-problems x1 , x2 , … xk
   for i from 1 to k do
            si = divide-and-conquer(xi)
   endfor
   combine the si solutions into solution s of x
   return (s)
  endif
end
```

Some applications

- Find the maximum and minimum values
- Multiply two integers
- Multiply two matrices
- Quicksort

Find the maximum and minimum values

- Problem
  - Find the maximum value (max) and minimum value (min) in a list A[1..n]. How many comparisons are needed between the elements of A?
- Brute force algorithm

```
maxmin(A, n)
begin
   max = A[1]
   min = A[1]
   for i from 2 to n
      if (A[i] > max) then max = A[i]
      endif
      if (A[i] < min) then min = A[i]
      endif
   endfor
   return (max, min)
end
```

Find the maximum and minimum values

- Divide and conquer algorithm
  - Divide the array into two sub-arrays/halves
  - Conquer sub-solutions
    - Apply the idea to each half
  - Combine the results
    - The maximum and minimum values of the array are obtained by comparing the maximum and minimum values of two halves

Find the maximum and minimum values

- Divide and conquer algorithm

```
maxmin(A,x,y)
begin
   if (y-x ≤ 1 ) then
      return (max(A[x], A[y])), min(A[x], A[y]))
   else
      (max1, min1) = maxmin(A, x, ⌊(x+y)/2⌋ )
      (max2, min2) = maxmin(A, ⌊(x+y)/2⌋+1, y )
      return (max(max1, max2), min(min1, min2))
   endif
end
```

Find the maximum and minimum values

- Algorithm analysis
  - Calculate the number of comparisons
    - C(n): number of comparisons, with n = y – x + 1
    - Suppose n is a power of 2, that means y–x is odd and x+y is also odd
    - Size of sub-problems

$$\lfloor (x+y)/2 \rfloor - x + 1 = \frac{x+y-1}{2} - x + 1 = \frac{y-x+1}{2} = \frac{n}{2}$$

$$y - (\lfloor (x+y)/2 \rfloor + 1) + 1 = y - \frac{x+y-1}{2} = \frac{y-x+1}{2} = \frac{n}{2}$$

    - So

$$C(n) = \begin{cases} 1 & if\ n = 2 \\ 2C(n/2) + 2 & else \end{cases}$$

Find the maximum and minimum values

- Algorithm analysis
  - Calculate the number of comparisons

$$
\begin{aligned}
C(n) &= 2C(n/2)+2 \\
&= 2^2 C(n/4)+2^2+2 \\
&= 2^3 C(n/8)+2^3+2^2+2 \\
&= 2^i C(n/2^i)+\sum_{k=1}^{i} 2^k = 2^{\log n-1} C(2)+\sum_{k=1}^{\log n-1} 2^k \\
&= 2^{\log n-1}+\frac{1-2^{\log n-1+1}}{1-2} = \frac{2^{\log n}}{2}+2^{\log n}-1 = \frac{3}{2}n-1
\end{aligned}
$$

→ The divide and conquer algorithm uses only 75% of the comparisons compared to the brute force algorithm

Multiply two integers

- Idea
  - The number of digits of two integers must be equal and must be equal to the power of 2
    - If the number of digits of two integers is different, add zeros to the left
  - Idea: we replace the multiplication of two n-digit integers by 4 multiplications of two n/2-digit integers. Next, replace the multiplication of two n/2-digit integers by 4 multiplications of two n/4-digit integers…
  - We need to multiply two integers: a and b. Let a_L and a_R be the left and right halves of the number a, similarly b_L and b_R be the left and right halves of b

Multiply two integers

- Idea

$$
\begin{array}{rrr}
 & a_L & a_R \\
* & b_L & b_R \\
\hline
 & a_L b_R & a_R b_R \\
+ \; a_L b_L & a_R b_L & \\
\hline
\end{array}
$$

$$a_L b_L + (a_L b_R + a_R b_L) + a_R b_R$$

$$
\begin{aligned}
a * b &= (a_L * 10^{n/2} + a_R)*(b_L * 10^{n/2} + b_R) \\
&= a_L * b_L * 10^n + a_L * b_R * 10^{n/2} + a_R * b_L * 10^{n/2} + a_R * b_R \\
&= a_L * b_L * 10^n + (a_L * b_R + a_R * b_L)10^{n/2} + a_R * b_R
\end{aligned}
$$

To multiply a and b by the halves of a and b, four multiplications are required.

Multiply two integers

- Design algorithm
  - **Divide**: Divide each *n*-digit integer into two *n*/2-digit integers.
  - **Conquer:** Perform multiplication of two *n/2*-digit integers by recursive calls.
  - **Combination**: Perform addition of two integers with *n*/2-digits.

Multiply two integers

- The divide and conquer algorithm

```
MultiplyIntegers(a, b, n)
begin
   if (n = 1) then
      return a × b
   else
         Divide a into two halves a_L and a_R , b into two halves b_L and b_R
         p_1 ← MultiplyIntegers(a_L , b_L , n/2)
         p_2 ← MultiplyIntegers(a_L , b_R , n/2)
         p_3 ← MultiplyIntegers(a_R , b_L , n/2)
         p_4 ← MultiplyIntegers(a_R , b_R , n/2)
         return (p_1 × 10^n + (p_2 + p_3 ) × 10^(n/2) + p_4 )
   end if
end
```

Multiply two integers

- Running time

$$C(n) = \begin{cases} c & if\ \ n = 1 \\ 4C(n/2) + dn & if\ \ n > 1 \end{cases}$$

$$\begin{aligned} C(n) &= 4C(n/2) + dn \\ &= 4^2 C(n/4) + 4d(n/2) + dn \\ &= 4^3 C(n/8) + 2^2 dn + 2dn + dn \\ &= 4^i C(n/2^i) + dn \sum_{k=0}^{i-1} 2^k \\ &= 4^{\log n} C(1) + dn \sum_{k=0}^{\log n - 1} 2^k \\ &= cn^2 + dn \frac{1 - 2^{\log n - 1 + 1}}{1 - 2} \\ &= cn^2 + dn^2 - dn. \end{aligned}$$

Multiply two integers

- The Karatsuba algorithm
  - Reduce the number of multiplications of a and b by the halves of a and b from 4 to 3

Let: $p = a_L * b_L$

$q = a_R * b_R$

$r = (a_L + a_R) * (b_L + b_R)$

Then:

$$a * b = a_L * b_L * 10^n + (a_L * b_R + a_R * b_L) * 10^{n/2} + a_R * b_R$$

$$= p * 10^n + (r - p - q) * 10^{n/2} + q$$

Multiply two integers

- The Karatsuba algorithm

```
Karatsuba(a, b, n)
begin
   if (n = 1) then
      return a × b
   else
         Divide a into two halves a_L and a_R , b into two halves b_L and b_R
         p ← Karatsuba(a_L , b_L , n/2)
         q ← Karatsuba(a_L , b_R , n/2)
         r ← Karatsuba(a_R + a_L, b_R + b_L , n/2)
         return (p × 10^n + (r – p – q) × 10^(n/2) + q)
   end if
end
```

Multiply two integers

- The Karatsuba algorithm
  - Running time

$$C(n) = \begin{cases} c & if\ \ n = 1 \\ 3C(n/2) + dn & if\ \ n > 1 \end{cases}$$

$$O(n^{\log 3}) = O(n^{1.59})$$

$$C(n) = 3C(n/2) + dn$$

$$= 3^2 C(\frac{n}{4}) + \frac{3}{2}dn + dn$$

$$= 3^i C(\frac{n}{2^i}) + dn \sum_{k=0}^{i-1} \left(\frac{3}{2}\right)^k$$

$$= 3^{\log n} C(1) + dn \sum_{k=0}^{\log n - 1} \left(\frac{3}{2}\right)^k$$

$$= cn^{\log 3} + dn \frac{1 - (3/2)^{\log n - 1 + 1}}{1 - 3/2}$$

$$= cn^{\log 3} + 2dnn^{\log 3/2} - 2dn$$

$$= cn^{\log 3} + 2dnn^{\log 3 - \log 2} - 2dn$$

$$= cn^{\log 3} + 2dn \frac{n^{\log 3}}{n} - 2dn$$

$$= cn^{\log 3} + 2dn^{\log 3} - 2dn.$$

Multiply two matrices

- Problem
  - Multiply two n by n matrices: C = A.B
- Brute force algorithm

```
matrixproduct(A, B, n)
begin
   for i from 1 to n
      for j from 1 to n
         C(i,j) = 0
         for k from 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
         endfor
      endfor
   endfor
   return (C)
end
```

Algorithm that performs $O(n^3)$ additions and multiplications

Multiply two matrices

- Divide and conquer algorithm (1)
  - Suppose $n = 2^k$
  - Divide matrices A, B, C into matrices of size n/2, then C = A.B respectively

$$\begin{pmatrix} r & s \\ t & u \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix}$$

  - From there, we have

    r = ae + bg

    s = af + bh

    t = ce + dg

    u = cf + dh

Multiply two matrices

- Divide and conquer algorithm (2)

```
matrixproduct(A, B, n)
begin
   if (n = 1) then return (A.B)
   else
      Divide A and B into 8 matrices of size n/2: a, b, …, h
      r = matrixproduct(a, e, n/2) + matrixproduct(b, g, n/2)
      s = matrixproduct(a, f, n/2) + matrixproduct(b, h, n/2)
      t = matrixproduct(c, e, n/2) + matrixproduct(d, g, n/2)
      u = matrixproduct(c, f, n/2) + matrixproduct(d, h, n/2)
      Combining r, s, t and u gives matrix C
       return C
   end if
end
```

Complexity: $C(n) = 8C(n/2) + n^2$, $C(1) = 1$

where n^2 is the number of additions

Multiply two matrices

- Divide and conquer algorithm (3)

$$
\begin{aligned}
C(n) &= 8C(n/2) + n^2 \\
&= 8(8C(n/4) + (n/2)^2) + n^2 \\
&= 8^2 C(n/4) + 2n^2 + n^2 \\
&= 8^i C(n/2^i) + n^2 \sum_{k=0}^{i-1} 2^k = 8^{\log n} C(1) + n^2 \sum_{k=0}^{\log n - 1} 2^k \\
&= 8^{\log n} + n^2 \frac{1 - 2^{\log n - 1 + 1}}{1 - 2} = 8^{\log n} + n^2 (2^{\log n} - 1) = \\
&= 2^{3\log n} + n^2 (n-1) = n^3 + n^2 (n-1) = 2n^3 - n^2
\end{aligned}
$$

$\Rightarrow$ $\boxed{O(n^3)}$

Multiply two matrices

- The Strassen algorithm (1)

  $m_1 = (a+c)(e+f)$

  $m_2 = (b+d)(g+h)$

  $m_3 = (ad)(e+h)$

  $m_4 = a(fh)$

  $m_5 = (c+d)e$

  $m_6 = (a+b)h$

  $m_7 = d(ge)$

  - Then

    $r = m_2 + m_3 - m_6 - m_7$

    $s = m_4 + m_6$

    $t = m_5 + m_7$

    $u = m_1 - m_3 - m_4 - m_5$

+ Perform only 7 matrix multiplications compared to 8 matrix multiplications of the divide and conquer algorithm

+ Perform more additions and subtractions matrix than the divide and conquer algorithm

Multiply two matrices

- The Strassen algorithm (2)
  - Why correct?

$$
\begin{aligned}
r &= m_2 + m_3 - m_6 - m_7 \\
&= (b+d)(g+h) + (ad)(e+h) - (a+b)h - d(ge) \\
&= bg+bh+dg+dh+ae+ah-de-dh-ah-bh-dg+de \\
&= ae + bg
\end{aligned}
$$

$$s = m_4 + m_6 = a(fh) + (a+b)h = af + bh$$

$$t = m_5 + m_7 = (c+d)e + d(ge) = ce + dg$$

$$
\begin{aligned}
u &= m_1 - m_3 - m_4 - m_5 \\
&= (a+c)(e+f) - (ad)(e+h) - a(fh) - (c+d)e \\
&= ae+af+ce+cf-ae-ah+de+dh-af+ah-ce-de \\
&= cf + dh
\end{aligned}
$$

Multiply two matrices

- The Strassen algorithm (3)

```
matrixproduct(A, B, n)
begin
   if (n = 1) then return (A.B)
   else
            Divide A and B into 8 matrices of size n/2: a, b, …, h
            m1 = matrixproduct(a+c, e+f, n/2)
            m2 = matrixproduct(b+d, g+h, n/2)
            m3 = matrixproduct(ad, e+h, n/2)
            m4 = matrixproduct(a, fh, n/2)
            m5 = matrixproduct(c+d, e, n/2)
            m6 = matrixproduct(a+b, h, n/2)
            m7 = matrixproduct(d, ge, n/2)
            r = m2 + m3 - m6 - m7
            s = m4 + m6
            t = m5 + m7
            u = m1 - m3 - m4 - m5
   end if
end
```

Multiply two matrices

- The Strassen algorithm (4)
  - Complexity

    $C(n) = 7C(n/2) + 18n^2/4$

    $C(1) = 1$

Multiply two matrices

- The Strassen algorithm (5)
  - Complexity

$$
\begin{aligned}
C(n) &= 7C(n/2) + \frac{9}{2}n^2 \\
&= 7(7C(n/4) + \frac{9}{2}(n/2)^2) + \frac{9}{2}n^2 \\
&= 7^2 C(n/8) + \frac{9}{2}7n^2/4 + \frac{9}{2}n^2 \\
&= 7^i C(n/2^i) + \frac{9}{2}n^2 \sum_{k=0}^{i-1}\left(\frac{7}{4}\right)^k = 7^{\log n} C(1) + \frac{9}{2}n^2 \sum_{k=0}^{\log n - 1}\left(\frac{7}{4}\right)^k \\
&= 7^{\log n} + \frac{9}{2}n^2 \frac{(7/4)^{\log n - 1 + 1} - 1}{7/4 - 1} = 7^{\log n} + \frac{9}{2}n^2 \frac{((7/4)^{\log n} - 1)}{3/4} = \\
&= n^{\log 7} + 6n^2((7/4)^{\log n} - 1) = n^{\log 7} + 6n^2(n^{\log 7 - \log 4} - 1) = n^{\log 7} + 6n^2\left(\frac{n^{\log 7}}{n^2} - 1\right) \\
&= O(n^{\log 7}) = O(n^{2.8})
\end{aligned}
$$

Quicksort

- Divide and conquer algorithm

```
quicksort (A)
begin
   if (n = 1) then return (A)
   else
         Select an element x in list A
         Split list A into A1 , A2 , A3 such that the elements of A1 is
         less than x, the elements of A2 are equal to x, and the
         elements of A3 is greater than x
         return (quicksort(A1), A2 , quicksort(A3))
   end if
end
```

Quicksort

- Worst case complexity
  - The list A is sorted and we choose x as the first element

$$C(n) = \begin{cases} 0 & if\ n \le 1 \\ C(n-1) + n - 1 & else \end{cases}$$

  - So: $\Theta(n^2)$

Quicksort

- Best case complexity
  - The selected element x is always the median value, i.e. split into two sublists of size ~ n/2, i.e. i=n/2

$$C(n) = \begin{cases} 0 & if\ n \le 1 \\ 2C(n/2) + n - 1 & else \end{cases}$$

  - So: O(nlog(n))

Quicksort

- Average case complexity (1)
  - C(n): number of comparisons when sorting list A with n different elements
  - C(0) = C(1) = 0
  - Suppose x is the i-th smallest element in list A

    $|A_1| = i-1$

    $|A_2| = 1$

    $|A_3| = n-i$
  - Suppose the probability of the i_{th} smallest element x in A is 1/n

Quicksort

- Average case complexity (2)
  - The recursive call takes average time C(i-1) and C(n-i), with i $\in$ [1..n] with probability 1/n
  - To divide A into A_1 , A_2 and A_3 requires n-1 comparisons
  - So, with n $\geq$ 2

$$C(n) = \frac{1}{n}\sum_{i=1}^{n}\left(C(i-1)+C(n-i)\right)+n-1$$

  - But

$$\sum_{i=1}^{n}\left(C(i-1)+C(n-i)\right)=\sum_{i=1}^{n}C(i-1)+\sum_{i=1}^{n}C(n-i)=\sum_{i=0}^{n-1}C(i)+\sum_{i=0}^{n-1}C(i)=2\sum_{i=2}^{n-1}C(i)$$

  - So, with n $\geq$ 2

$$C(n) = \frac{2}{n}\sum_{i=2}^{n-1}C(i)+n-1 \qquad (1)$$

Quicksort

- Average case complexity (3)
  - Multiply both sides of (1) by n, with n ≥ 2 we have

$$nC(n) = 2\sum_{i=2}^{n-1} C(i) + n^2 - n \qquad (2)$$

  - Replace n by n-1, with n ≥ 3 we have

$$(n-1)C(n-1) = 2\sum_{i=2}^{n-2} C(i) + n^2 - 3n + 2 \qquad (3)$$

  - Take (2) minus (3)

$$nC(n) - (n-1)C(n-1) = 2C(n-1) + 2(n-1)$$

  - So

$$nC(n) = (n+1)C(n-1) + 2(n-1) \qquad (4)$$

  - Divide both sides of (4) by n(n+1)

$$C(n)/(n+1) = C(n-1)/n + 2(n-1)/n(n+1) \qquad (5)$$

Quicksort

- Average case complexity (4)
  - Let S(n) = C(n)/(n+1), from (5) we have, with n ≥3

$$S(n) = S(n-1) + 2(n-1)/n(n+1) \qquad (6)$$

with S(0) = S(1) = 0

(5) is true even when n = 2, S(2)=C(2)/3=1/3

  - So

$$S(n) \le \begin{cases} 0 & if\ n \le 1 \\ S(n-1) + 2/n & else \end{cases}$$

$$S(n) \le S(n-1) + 2/n \le S(n-2) + 2/(n-1) + 2/n$$

$$\le S(n-3) + 2/(n-2) + 2/(n-1) + 2/n$$

$$\le S(n-i) + 2\sum_{k=n-i+1}^{n} 1/k$$

Quicksort

- Average case complexity (5)
  - Substitute i = n-1, we have

$$S(n) \le S(1) + 2\sum_{k=2}^{n} \frac{1}{k} = 2\sum_{k=2}^{n} \frac{1}{k} \le 2\int_{1}^{n} \frac{1}{x}\,dx = 2\ln n$$

  - So

$$\begin{aligned} C(n) &= (n+1)S(n) \\ &\le 2(n+1)\ln n \\ &\le 2(n+1)\frac{\log n}{\log e} \\ &\le 1.386(n+1)\log n \end{aligned}$$

  - So O(nlog(n))

Exercises

- Problem 1
  - Design a divide and conquer algorithm to compute the sum of an array of numbers.
  - Evaluate the running time of the algorithm.

- Problem 2
  - Prove that it is possible to multiply two polynomials ax+b and cx+d with only 3 multiplications (*hint: one of the multiplications (a+b)(c+d)*)

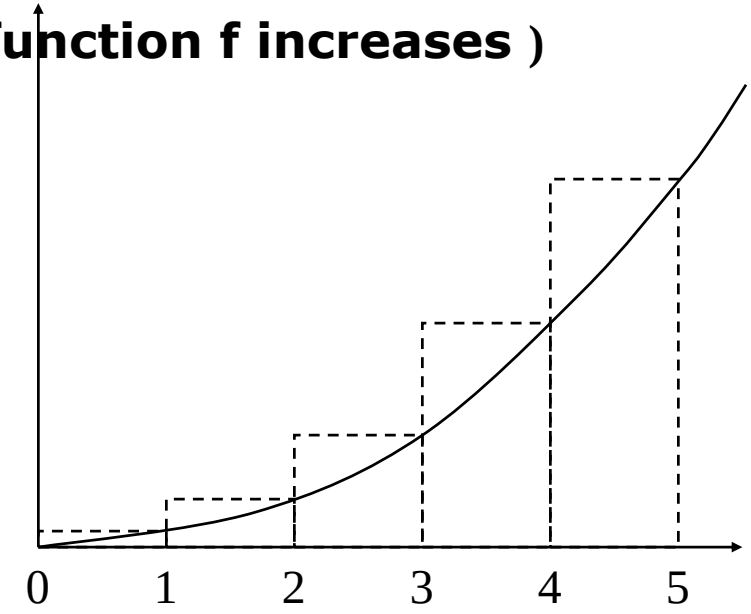
Exercises

- Problem 3
  1. Construct two divide-and-conquer algorithms to multiply two polynomials with complexity $O(n^{\log_2^3})$
     a. The first algorithm requires splitting the polynomial into two polynomials, one half of degree n/2 (power [0..n/2]) and the other half of degree n (power [n/2+1..n])
     b. The second algorithm requires splitting the polynomial into two polynomials, one half with an even exponent, the other half with an odd exponent
  2. Prove that two integers represented by n bits can be multiplied by an algorithm of complexity $O(n^{\log_2^3})$

Sum

$$F(n) = \sum_{i=1}^{n} f(i)$$

(**function f increases**)



For example: $f(x)=i^p$

$$\int_0^5 f(t)\,dt \le \sum_{i=1}^{5} f(i)$$

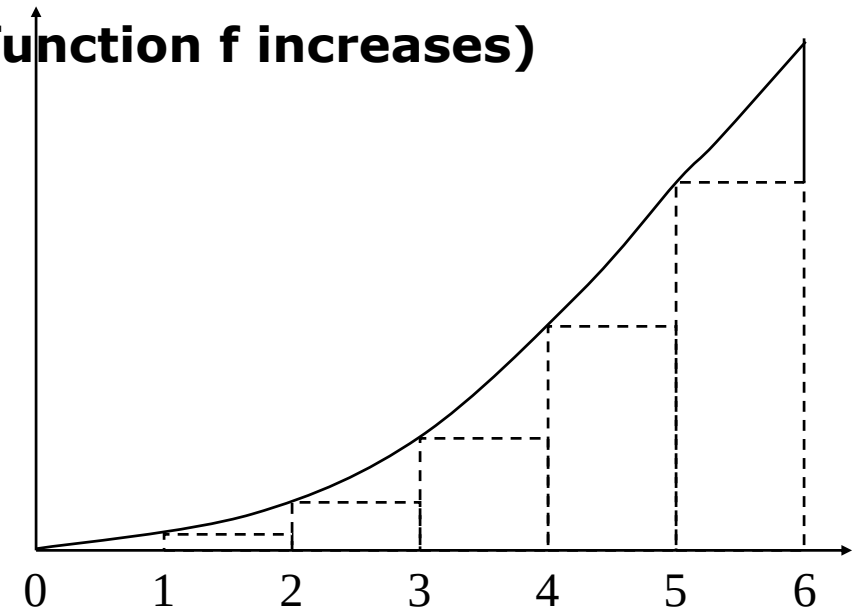
Overview

$$\int_0^n f(t)\,dt \le \sum_{i=1}^{n} f(i)$$

Sum

- Sum (2)

(function f increases)



$$\sum_{i=1}^{5} f(i) \le \int_{1}^{6} f(t)dt$$

Overview

$$\sum_{i=1}^{n} f(i) \le \int_{1}^{n+1} f(t)dt$$

So :

$$\int_{0}^{n} f(t)dt \le \sum_{i=1}^{n} f(i) \le \int_{1}^{n+1} f(t)dt$$

If the function f decreases:

$$\int_{1}^{n+1} f(t)dt \le \sum_{i=1}^{n} f(i) \le \int_{0}^{n} f(t)dt$$